



AERIALCAST – DRONE TELEMETRY SYSTEM

Created on: 17 - 05 - 2026

aerialcast.irc-enter.tech

Created by: Muhammad Okta Toriq Gunawan

status of the document: Develepoment

Versions and signoff assertions:**Test Plan History:**

Version Num	Date	Author	Changes Summary
1.0	17 - 05 - 2026	Muhammad Okta Toriq Gunawan	Initial Planning

Changes Approval:

Name	Approved	Date

SRS Document Reference:

Document Name	URL
Business Requirements	
Technical Requirements	

Daftar Isi:

1. INTRODUCTION:	3
1.1Project Overview:	3
1.2Audience:	3
2. TEST STRATEGY:	5
2.1Test Objectives:	5
2.2Test Assumptions:	5
2.3Test Principles:	6
2.4Scope and Levels of Testing:	7
2.4.1 Build Acceptance:	7
2.4.2 Functional Test:	7
2.5LOE (Level OF Effort):	9
3. EXECUTION STRATEGY:	11
3.1Start and Exit Criteria:	11
3.2Test Cycle:	12
3.3Test Metrics:	12
3.4Defect tracking & Reporting:	13
4. TEST MANAGEMENT PROCESS:	15
4.1Test Design Process:	15
4.2Test Execution Process:	15
4.3Test Risks & Dependencies:	16
4.4Team Members Roles:	16
5. TEST ENVIRONMENT:	18
5.1Test Stage:	18
5.1Test Compatibility:	18
6. DEADLINES AND DELIVERABLES:	19
BEST OF LUCK TEAM!	20

1.INTRODUCTION:

1.1 Project Overview:

Dokumen ini menguraikan Rencana Pengujian (Test Plan) untuk sistem AerialCast, yaitu sebuah sistem telemetri Unmanned Aerial Vehicle (UAV) jarak jauh berbasis Internet of Things (IoT). Arsitektur sistem ini menggunakan komunikasi LoRa untuk mentransmisikan data penerbangan menuju Ground Control Station (GCS), yang kemudian diteruskan ke layanan backend berbasis Flask. Data telemetri tersebut disimpan ke dalam basis data timeseries menggunakan TimescaleDB (PostgreSQL) pada platform Neon, untuk selanjutnya divisualisasikan secara real-time melalui antarmuka pengguna (frontend) yang dibangun dengan Next.js dan TypeScript.

Fokus utama dari dokumen pengujian ini adalah pelaksanaan Automated Testing dan Integration Testing pada lapisan aplikasi web. Pengujian otomatis akan dilakukan menggunakan kerangka kerja Selenium untuk memvalidasi perilaku antarmuka pengguna (UI) secara end-to-end (E2E). Dikarenakan pengujian berfokus pada perangkat lunak, aliran data sensor IoT dari perangkat keras fisik akan disimulasikan melalui mocking data untuk memastikan sistem merespons perubahan matriks telemetri sesuai dengan skenario yang telah ditentukan.

1.2 Audience:

Dokumen ini ditujukan kepada pihak-pihak yang terlibat dalam proses pengembangan, pengujian, serta evaluasi kualitas dari sistem AerialCast. Silakan merujuk pada tabel di bawah ini untuk mengetahui daftar audiens yang berkepentingan:

Title/Department	Name	Deskripsi Peran
Quality Assurance / Tester	Muhammad Okta Toriq Gunawan	Bertanggung jawab dalam merancang skenario, menulis skrip otomatis Selenium, dan mengeksekusi pengujian.

Backend & IoT Developer	Fadhail	Bertanggung jawab memastikan ketersediaan API dari Flask untuk simulasi integrasi data.
Academic Supervisor	Bpk.Roni Andarsyah, S.T., M.Kom., SFPC	Bertindak sebagai peninjau untuk mengevaluasi kelayakan metode pengujian Software Quality Assurance (QA) yang diterapkan pada sistem.

2. TEST STRATEGY:

2.1 Test Objectives:

Objektif utama dari proses pengujian pada sistem AerialCast adalah sebagai berikut:

- a) Memvalidasi fungsionalitas antarmuka pengguna (user interface) pada dashboard Next.js secara otomatis menggunakan Selenium untuk memastikan seluruh komponen visual berjalan sesuai spesifikasi.
- b) Memastikan integrasi aliran data (data pipeline) berjalan lancar dari lapisan backend Flask, penyimpanan timeseries pada TimescaleDB, hingga sinkronisasi data ke lapisan frontend.
- c) Memverifikasi kemampuan antarmuka dalam menampilkan pembaruan data telemetry (koordinat, sinyal, dan kecepatan) secara real-time tanpa adanya gejala freeze, crash, atau keterlambatan (latency) yang masif pada peramban (browser).
- d) Menemukan dan mendokumentasikan cacat sistem (defect/bug) pada lapisan aplikasi web sedini mungkin sebelum sistem masuk ke tahap rilis akhir.

2.2 Test Assumptions:

Pengujian ini dirancang dan dilaksanakan berdasarkan beberapa asumsi berikut:

- a) Perangkat keras fisik Ground Control Station (GCS) dan drone diasumsikan tidak tersedia pada lingkungan pengujian, sehingga perannya digantikan sepenuhnya oleh skrip simulator (data generator) eksternal.
- b) Skrip simulator telemetry diasumsikan mampu menghasilkan dan mengirimkan struktur serta format paket data JSON yang identik 100% dengan data asli dari modul LoRa.
- c) Lingkungan pengujian lokal (localhost) diasumsikan memiliki performa dan konfigurasi yang stabil serta mencukupi untuk menjalankan Flask, Next.js, basis data PostgreSQL, dan Selenium WebDriver secara simultan.

2.3 Test Principles

Strategi pengujian pada proyek ini berpegang pada prinsip-prinsip berikut:

- a) Otomasi Berbasis UI (UI Automation-First): Pengujian fungsionalitas dashboard dan regresi antarmuka diprioritaskan menggunakan skrip otomasi Selenium guna meminimalkan kesalahan manusia (human error) dan mempercepat siklus uji.
- b) Isolasi Lingkungan (Environment Isolation): Seluruh rangkaian pengujian wajib diisolasi dan dijalankan hanya pada lingkungan lokal atau staging untuk mencegah manipulasi atau kerusakan data pada basis data produksi.
- c) Pengujian Berorientasi Kasus Ekstrem (Edge-Case Oriented): Pengujian tidak hanya berfokus pada kondisi normal (happy path), melainkan juga harus menguji skenario anomali, seperti pengiriman data kosong (null) atau pemutusan simulasi data secara mendadak untuk melihat ketahanan respons UI.

2.4 Scope and Levels of Testing:

2.4.1 Build Acceptance:

- a) **Objective of the test:** Memastikan bahwa kode sumber aplikasi, baik layanan backend (Flask) maupun frontend (Next.js), dapat dikompilasi (build) dan dijalankan di lingkungan lokal tanpa mengalami hambatan kritis (fatal error) sebelum pengujian lebih lanjut dilakukan.
- b) **When:** Dilakukan setiap kali terdapat pembaruan kode pada repository atau tepat sebelum skrip pengujian otomatisasi dijalankan.
- c) **Responsibility:** Quality Assurance / Tester.
- d) **Methodology:** Pengujian manual (Smoke Testing) dengan mengeksekusi perintah build serta memverifikasi bahwa seluruh layanan (services) dan basis data lokal dapat diakses pada porta (port) yang telah ditetapkan.

2.4.2 Functional Test:

- a) **Objective of the test:** Memvalidasi fungsionalitas antarmuka pengguna sistem AerialCast dan memastikan integrasi aliran data bekerja dengan baik. Pengujian mencakup validasi pembaruan elemen visual di dasbor secara real-time saat menerima data telemetri.
- b) **When:** Dilaksanakan segera setelah tahap Build Acceptance dinyatakan berhasil dan stabil.
- c) **Responsibility:** Quality Assurance / Tester.
- d) **Methodology:** Automated E2E (End-to-End) Testing menggunakan framework Selenium WebDriver dengan bahasa pemrograman Python. Pengujian ini dipadukan dengan implementasi Data Simulator untuk menyuntikkan (inject) respons data buatan (GPS, Sinyal, dan kecepatan) guna merekayasa pergerakan drone.

*Test Deliverables:

What to Deliver	Responsibility	Review By
Test Plan Doc.	Quality Assurance / Tester	Academic Supervisor
Functional Test Cases	Quality Assurance / Tester	Academic Supervisor
Defects Logging Under	Quality Assurance / Tester	Academic Supervisor
Closure Report	Quality Assurance / Tester	Academic Supervisor

2.4.3 User Acceptance Test (UAT):

- a) **Objective of the test:** Memastikan bahwa sistem visualisasi GCS secara keseluruhan telah memenuhi standar spesifikasi, metrik performa UI, dan kebutuhan fungsional yang ditetapkan pada awal proyek pengembangan.
- b) **When:** Dilakukan pada tahap akhir, setelah seluruh rangkaian Functional Test selesai dieksekusi dan semua temuan cacat sistem (bug) yang bersifat kritis telah diperbaiki.
- c) **Responsibility:** Academic Supervisor / Reviewer (sebagai representasi pemangku kepentingan).
- d) **Methodology:** Peninjauan kelayakan (Acceptance Review) berdasarkan hasil laporan pengujian otomatis Selenium, serta evaluasi manual secara langsung pada dasbor antarmuka sistem.

*Test Deliverables:

What to Deliver	Responsibility	Review By
UAT Test Cases	Quality Assurance / Tester	Academic Supervisor
Users Support	Quality Assurance / Tester	Academic Supervisor

2.5 LOE (Level OF Effort):

Tabel berikut menguraikan estimasi beban kerja dan alokasi waktu untuk setiap fase dalam siklus pengujian perangkat lunak sistem AerialCast:

QA Task	Working Hours / Days	Note(s)
SRS meeting(s)	2 Jam	Analisis struktur aliran data telemetri dan spesifikasi UI dasbor.
Requirements Analysis (QA meeting)	2 Jam	
Project Test Plan	6 Jam	Isolasi dan konfigurasi <i>localhost</i> untuk Flask, Next.js, dan skrip simulator data.
Testing Environment Setup (Hardware)	5 Jam	Isolasi dan konfigurasi <i>localhost</i> untuk Flask, Next.js, dan skrip simulator data.
Ad-hoc testing round once deploy	2 Jam	Pengecekan stabilitas awal layanan lokal sebelum otomasi dijalankan.
Test Cases Development including reviews	8 Jam	Perancangan skenario pengujian E2E dan <i>Integration</i> secara terstruktur.
Total Test Design Time	25 Jam	Akumulasi waktu fase perancangan.
Test Execution		
Automation test scripts	12 Jam	Penulisan, uji coba, dan penyempurnaan skrip otomasi berbasis Python Selenium.
First testing round	4 Jam	Eksekusi skrip awal pada dasbor antarmuka.

Results logged into Tracker/Report	1 Jam	Pencatatan temuan cacat sistem (bug) atau anomali respons.
Results logged into JIRA	1 Jam	Pembaruan status resolusi temuan.
Second testing round	3 Jam	Verifikasi ulang setelah tahap perbaikan dilakukan pada basis kode.
Final Sanity round form the exit report	2 Jam	Evaluasi stabilitas rilis kandidat sebelum pelaporan akhir.
Total Test Execution Time	22 Jam	Akumulasi waktu fase eksekusi pengujian.
UAT Phase		
UAT test	2 Jam	Peninjauan hasil eksekusi Selenium bersama Supervisor Akademik.
Support	1 Jam	Pendampingan teknis dan dokumentasi selama masa evaluasi.
Total UAT Time	3 Jam	Akumulasi waktu UAT.
Margin (for all Phases)	5 Jam	Alokasi waktu penyangga (buffer) untuk menanggulangi kendala teknis tak terduga.
<u>Total Testing Time (Working Hours / Days)</u>	55 Jam	Estimasi kumulatif keseluruhan siklus QA.

3. EXECUTION STRATEGY:

3.1 Start and Exit Criteria:

Pengujian dapat dimulai apabila prasyarat berikut telah terpenuhi:

- a) Lingkungan pengujian lokal (localhost) telah diatur dan seluruh layanan (basis data PostgreSQL, backend Flask, dan frontend Next.js) beroperasi tanpa galat (error).
- b) Skrip simulator data telemetri (mocking data) telah selesai dikonfigurasi dan mampu menyuntikkan muatan (payload) JSON secara periodik ke titik akhir (endpoint) API.
- c) Kerangka kerja otomasi Selenium WebDriver telah diinstalasi, dikonfigurasi, dan terhubung dengan sukses ke peramban (browser) target pengujian.
- d) Rencana pengujian dan skenario kasus uji (test cases) telah didokumentasikan dan disetujui.

Proses pengujian dinyatakan selesai dan siap untuk diserahkan apabila kriteria berikut tercapai:

- a) Seluruh rangkaian skrip pengujian otomasi Selenium telah dieksekusi hingga tuntas.
- b) Tidak ditemukan cacat sistem (defect/bug) dengan tingkat keparahan "Kritis" (Critical) maupun "Tinggi" (High) pada antarmuka dasbor.
- c) Laporan eksekusi pengujian beserta matriks pencatatan bug telah diperbarui secara menyeluruh.

Adapun kondisi spesifik untuk kriteria keluar (exit criteria) ditetapkan sebagai berikut:

Condition	Result
Seluruh skrip pengujian Selenium dieksekusi 100% pada lingkungan lokal.	<i>Passed</i>
Tingkat keberhasilan eksekusi pengujian (Pass Rate) mencapai minimal 95%.	<i>Passed</i>

Cacat sistem dengan tingkat prioritas Kritis dan Tinggi telah diselesaikan dan ditutup secara resmi.	<i>Passed</i>
Waktu respons pembaruan komponen visual di antarmuka berada di bawah batas toleransi latensi aplikasi web standar.	<i>Passed</i>
Aliran data telemetri yang disimulasikan berhasil divisualisasikan tanpa galat sinkronisasi UI.	<i>Passed</i>

3.2 Test Cycle:

Siklus pengujian pada proyek ini dibagi menjadi tahapan iteratif untuk memastikan pemulihan sistem yang konsisten terhadap setiap temuan anomali fungsional. Siklus eksekusi tersebut meliputi:

- a) **Cycle 1:** Eksekusi menyeluruh terhadap semua kasus uji fungsional dan otomasi antarmuka. Tahap ini berfokus pada identifikasi awal cacat sistem yang muncul saat skrip data telemetri simulasi berinteraksi dengan layanan Next.js.
- b) **Cycle 2:** Eksekusi ulang pada kasus uji yang sebelumnya dinyatakan gagal (failed). Tahapan ini dilakukan setelah pengembang (developer) melakukan perbaikan perbaikan pada basis kode sumber.
- c) **Cycle 3:** Eksekusi pengujian terbatas guna memvalidasi fungsionalitas inti aplikasi secara singkat, untuk menjamin stabilitas kandidat rilis sebelum sistem dievaluasi pada tahap pengujian penerimaan pengguna (User Acceptance Testing).

3.3 Test Metrics:

Metrik pengujian diterapkan untuk mengukur tingkat keberhasilan eksekusi skrip otomasi, melacak kemajuan proses pengujian, serta mengevaluasi kualitas fungsional sistem secara keseluruhan. Berikut adalah rincian pelaporan metrik pengujian yang akan digunakan:

Report For	Report Description	Frequency
Test Execution Status (Overall)	Laporan status akumulatif dari skenario pengujian E2E dan <i>Integration</i> menggunakan Selenium.	Harian.
Test coverage	Laporan persentase area fungsional dasbor AerialCast dan titik akhir (endpoint) API yang telah terjangkau oleh pengujian.	Mingguan.
Coverage for Automation scripts	Laporan perkembangan rasio skrip otomatis yang berhasil diimplementasikan dibandingkan dengan total kasus uji yang dirancang.	Harian.

3.4 Defect tracking & Reporting:

Setiap temuan anomali fungsional atau cacat sistem (bug) yang teridentifikasi selama proses eksekusi Selenium akan didokumentasikan dan dilacak guna memastikan penyelesaian yang terukur. Silakan merujuk pada alur berikut yang merepresentasikan siklus hidup pelaporan *bug* (*BUG life cycle*) pada proyek ini:

- a) **Penemuan (Find/Improper Behavior):** Cacat sistem diidentifikasi selama pengujian otomatisasi (misalnya: UI dasbor Next.js gagal memperbarui data altitudo). Temuan ini dilaporkan dan dibuatkan tiket melalui perangkat pelacak isu (issue tracker).
- b) **Penugasan (Assign to Developer):** Tiket temuan diserahkan kepada pengembang utama (Lead Developer) untuk kemudian ditugaskan kepada pihak yang relevan guna proses analisis dan perbaikan kode.
- c) **Tinjauan Kode (Code Review):** Setelah perbaikan dilakukan, kode akan ditinjau. Apabila disetujui, perubahan akan diterapkan pada lingkungan pengujian (Deploy changes). Apabila ditolak, tiket dikembalikan kepada pengembang.
- d) **Pengujian Penerimaan (Acceptance Testing):** Tim Quality Assurance melakukan pengujian regresi (Regression Testing) untuk memverifikasi apakah perbaikan telah menyelesaikan anomali tanpa memunculkan kendala baru.

e) Resolusi Akhir: Jika hasil pengujian ulang dinyatakan lulus (Passed), status kandidat rilis dinyatakan siap (Mark Release Ready) untuk kemudian diterapkan ke lingkungan produksi (Release to LIVE). Sebaliknya, jika pengujian gagal, tiket akan dibuka kembali (Reopen).

4. TEST MANAGEMENT PROCESS:

4.1 Test Design Process:

Perancangan skenario pengujian bergantung pada beberapa faktor utama yang tercantum di bawah ini:

- a) Spesifikasi fungsional dari antarmuka dasbor Next.js dan struktur payload JSON yang dikirimkan oleh layanan backend Flask.
- b) Fokus pada pengujian ujung-ke-ujung (End-to-End Testing) menggunakan Selenium WebDriver untuk memvalidasi pembaruan elemen visual secara real-time.
- c) Identifikasi kasus batas (edge cases), seperti pengujian terhadap respons UI ketika skrip simulator mengirimkan data yang tidak valid (invalid format) atau mengalami gangguan koneksi.
- d) Ketersediaan lingkungan pengujian lokal (localhost) yang terisolasi dan terkonfigurasi dengan basis data TimescaleDB independen guna menghindari interferensi dengan data lingkungan produksi.

4.2 Test Execution Process:

Eksekusi pengujian secara aktual harus mencakup seluruh kasus uji yang telah didokumentasikan, di mana setiap eksekusinya meliputi langkah-langkah berikut:

- a) Inisialisasi lingkungan pengujian lokal, termasuk menjalankan peladen (server) pengembangan Flask dan Next.js secara simultan.
- b) Eksekusi skrip simulator (data mocking) untuk menyuntikkan aliran data telemetri tiruan ke titik akhir (endpoint) API secara berkala.
- c) Eksekusi skrip otomasi Selenium untuk melakukan interaksi dan verifikasi (assertion) terhadap perubahan status atau nilai pada antarmuka peramban.

- d) Pendokumentasian status kelulusan pengujian (pass/fail) secara otomatis beserta tangkapan layar (screenshot) pada perangkat pelacak isu apabila ditemukan kegagalan integrasi.

4.3 Test Risks & Dependencies:

Risk	Severity	Back-Up
Perubahan elemen DOM (Class/ID) pada Next.js yang menyebabkan kegagalan locator Selenium.	Medium	Mengimplementasikan atribut statis data-testid pada elemen komponen UI yang menjadi target pengujian.
Ketidaksesuaian struktur skrip simulator dengan pembaruan payload asli dari modul LoRa.	High	Menetapkan dokumentasi skema JSON yang dikunci (API Schema Versioning) selama fase pengujian berlangsung.
Latensi sistem lokal yang memicu timeout pada pengujian otomatis.	Low	Mengonfigurasi mekanisme Explicit Wait atau Fluent Wait di dalam skrip Selenium dibandingkan menggunakan Hard Sleep.
Kegagalan basis data timeseries pada lingkungan testing.	Medium	Menyediakan berkas dump SQL statis sebagai cadangan untuk pemulihan data uji secara instan.

4.4 Team Members Roles:

NAME	ROLE	TASKS
Muhammad Okta Toriq Gunawan	Quality Assurance / Tester	Merancang Test Plan, mengembangkan skrip otomatis pengujian E2E dengan Selenium, mengeksekusi pengujian, serta melacak cacat sistem (bug reporting).
Fadhail	Backend & IoT Developer	Mengembangkan layanan API Flask, mengonfigurasi skema penyimpanan TimescaleDB, serta memastikan ketersediaan titik akhir (endpoint) untuk integrasi skrip simulator. Mengembangkan layanan API Flask, mengonfigurasi skema

		penyimpanan TimescaleDB, serta memastikan ketersediaan titik akhir (endpoint) untuk integrasi skrip simulator.
--	--	--

5. TEST ENVIRONMENT:

5.1 Test Stage:

Pengujian sistem AerialCast akan dilaksanakan sepenuhnya pada lingkungan pengembangan lokal (localhost) dan lingkungan staging yang terisolasi. Lingkungan produksi tidak akan digunakan dalam siklus pengujian otomatisasi ini guna memastikan bahwa skrip simulator data tidak mencemari atau merusak integritas basis data timeseries operasional.

5.1 Test Compatibility:

Pengujian proyek ini akan dilakukan untuk memastikan kompatibilitas terhadap spesifikasi dan konfigurasi berikut:

- a) Peramban web Firefox versi stabil terbaru, yang akan bertindak sebagai target utama eksekusi Selenium WebDriver.
- b) Sistem operasi berbasis Linux atau yang mendukung eksekusi peladen pengembangan secara simultan untuk Node.js (lapisan frontend Next.js) dan Python (lapisan backend Flask).
- c) Lingkungan basis data PostgreSQL lokal yang dilengkapi dengan ekstensi TimescaleDB untuk keperluan penyimpanan data timeseries sementara selama pengujian.
- d) Skrip simulator data telemetri yang berjalan pada environment Python untuk merekayasa muatan data perangkat keras GCS.

Seluruh lingkungan pengujian beserta perangkat lunak prasyarat yang telah disebutkan di atas harus tersedia dan terkonfigurasi dengan baik untuk setiap pengujian sebelum siklus eksekusi dimulai.

6. DEADLINES AND DELIVERABLES:

Untuk rincian tenggat waktu dari setiap fase perancangan hingga eksekusi pengujian, silakan merujuk pada tabel di bawah ini:

Phase	Duration	Note(s)
Requirements	3 Hari kerja	Analisis kebutuhan integrasi aliran data telemetri, spesifikasi UI dasbor, dan penentuan alat pengujian.
Designing	4 Hari kerja	Penyusunan dokumen Rencana Pengujian (Test Plan) serta perancangan skenario kasus uji (test cases).
Developing	7 Hari kerja	Penulisan skrip otomasi menggunakan Selenium, penyiapan basis data lokal, dan konfigurasi skrip simulator data.
Testing & maintenance	5 Hari kerja	Eksekusi pengujian E2E, pencatatan temuan cacat sistem (bug logging), perbaikan kode, dan pengujian regresi.
UAT phase	2 Hari kerja	Peninjauan kelayakan antarmuka dan persetujuan evaluasi akhir bersama Supervisor Akademik.

Hope you get all the Info you wish

BEST OF LUCK TEAM!

Greetings From

QA Team